

Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 5: **Eloquent ORM dan Relasi**

Pemetaan Objek ke Tabel, Relasi Antar Model, dan Query yang Efisien

Yang Akan Kamu Pelajari

1. Menjelaskan konsep **ORM** dan bagaimana Eloquent memetakan class ke tabel basis data
2. Mendefinisikan **relasi** antar model (`hasOne` , `hasMany` , `belongsTo` , `belongsToMany`)
3. Memahami **eager loading** dan cara menghindari masalah **N+1**
4. Menerapkan **paginasi** untuk listing data berskala besar

Slide ini membahas konsep. Menerapkan relasi, eager loading, dan paginasi pada Simple POS dikerjakan di jobsheet praktikum.

Bagian 1

Tanpa Relasi vs Dengan Relasi Eloquent

Tanpa relasi Eloquent

- Setiap butuh data terkait, tulis ulang query join secara manual
- Rawan salah ketik nama kolom join, berulang di banyak tempat

Dengan relasi Eloquent

- Dideklarasikan sekali di model
- Dipakai di mana saja lewat properti atau method, tanpa menulis ulang join

Mirip pohon silsilah keluarga: garis keturunan digambar sekali, lalu siapa pun yang membaca pohon itu langsung tahu hubungannya, tanpa menelusuri dokumen satu per satu.

Konsep ORM: Memetakan Class ke Tabel

ORM (Object-Relational Mapping): teknik yang memetakan class dalam kode ke tabel di basis data, dan objek ke baris, sehingga operasi basis data ditulis lewat method dan properti object, bukan query SQL mentah.

Class (Model)



ORM



Tabel (baris)

- Eloquent memakai pola **Active Record**: satu class model merepresentasikan satu tabel, sekaligus menyediakan method untuk query, simpan, dan hapus datanya sendiri

Relasi: Menghubungkan Model

Relasi: metode pada model Eloquent yang mendeskripsikan bagaimana satu tabel terhubung ke tabel lain, dipakai layaknya properti biasa, meskipun sebenarnya menjalankan query di belakang layar.

```
class Author extends Model
{
    public function articles(): HasMany
    {
        return $this->hasMany(Article::class);
    }
}

class Article extends Model
{
    public function author(): BelongsTo
    {
        return $this->belongsTo(Author::class);
    }
}
```

authors (satu)



articles (banyak)

hasOne: Relasi Satu ke Satu

```
class Author extends Model
{
    public function profile(): HasOne
    {
        return $this->hasOne(Profile::class);
    }
}
```

- Mirip `hasMany` , tapi memastikan hanya ada **satu** baris terkait, bukan banyak
- Contoh: satu penulis punya satu profil, bukan beberapa

belongsToMany: Relasi Banyak ke Banyak

```
class Article extends Model
{
    public function tags(): BelongsToMany
    {
        return $this->belongsToMany(Tag::class);
    }
}
```

- Satu artikel bisa punya banyak tag, satu tag bisa dipakai banyak artikel
- Butuh tabel pivot di tengah, mis. `article_tag`, memakai konvensi nama: kedua nama model tunggal,urut abjad, dipisah garis bawah

Empat Jenis Relasi: Ringkasan

Relasi	Arti	Contoh
hasOne	Satu baris induk punya satu baris terkait	Author punya satu Profile
hasMany	Satu baris induk punya banyak baris terkait	Author punya banyak Article
belongsTo	Baris ini menunjuk balik ke satu baris induk	Article menunjuk ke satu Author
belongsToMany	Banyak ke banyak lewat tabel pivot	Article dan Tag saling terhubung

Nama Metode Relasi Bukan Sekadar Kosmetik

- Nama itu menjadi kunci yang dipakai di tiga tempat berbeda:
 - **Pemanggilan metode** `$author->articles()` : mengembalikan query builder, bisa dirangkai dengan `->where()` , `->orderBy()` , dst.
 - **Akses properti** `$author->articles` : langsung mengembalikan koleksi hasilnya
 - **String di dalam** `with('articles')` : memberi tahu Eloquent relasi mana yang harus dimuat lebih awal

Bagian 2

Masalah N+1

Lazy Loading: relasi baru dimuat saat properti itu benar-benar diakses, baris demi baris, sumber utama masalah N+1.

N+1: pola query bermasalah, satu query untuk daftar utama, ditambah satu query terpisah untuk setiap baris guna memuat relasinya.

- Menampilkan 15 artikel beserta nama penulisnya, tanpa eager loading: 1 query daftar + 15 query penulis = **16 query**

Pendekatan	Jumlah Query
Lazy loading (tanpa <code>with()</code>)	1 query daftar + N query relasi (N+1)
Eager loading (<code>with('author')</code>)	2 query total

Eager Loading

Eager Loading: teknik memuat relasi di muka lewat `with()` , dalam jumlah query yang tetap kecil, alih-alih memuatnya satu per satu saat baru dibutuhkan.

```
$articles = Article::with('author')  
    ->latest()  
    ->paginate(15);
```

Jumlah query lewat eager loading tidak bergantung pada berapa banyak baris ditampilkan, hanya pada berapa lapis relasi yang dimuat. Menampilkan 15 artikel atau 150 artikel dengan `with('author')` sama-sama memakan 2 query total, bukan 2 dikali jumlah baris.

Eager Loading Berjenjang: Notasi Titik

```
$authors = Author::with('articles.comments')->get();
```

- Memuat setiap author, relasi `articles` -nya, **dan** relasi `comments` di dalam tiap article sekaligus
- Tetap dalam jumlah query tetap, tidak bertambah seiring jumlah baris

Bagian 3

Paginasi

Paginasi: teknik membagi hasil query besar menjadi halaman-halaman kecil, mengembalikan hanya sebagian baris pada satu waktu beserta metadata jumlah halaman.

```
public function index()
{
    $articles = Article::with('author')
        ->orderBy('title')
        ->paginate(10);

    return view('articles.index', compact('articles'));
}
```

```
<!-- resources/views/articles/index.blade.php -->
<div class="mt-4">
    {{ $articles->links() }}
</div>
```


Kapan Pakai Paginasi

Pakai `paginate()` setiap kali jumlah baris berpotensi tumbuh tak terbatas seiring waktu, seperti daftar artikel atau riwayat komentar. `get()` biasa masih cukup untuk daftar yang memang kecil dan tetap kecil, seperti daftar tag pada dropdown filter.

Membuktikan Paginasi Bekerja

```
php artisan tinker
>>> Article::orderBy('title')->paginate(10)->pluck('id');
>>> Article::orderBy('title')->paginate(10, ['*'], 'page', 2)->pluck('id');
```

- Bandingkan hasil kedua perintah
- ID yang tumpang tindih antara halaman 1 dan halaman 2 berarti ada masalah pada `orderBy` atau datanya

Menerapkan pada Simple POS

- Konsep relasi, eager loading, dan paginasi ini akan kamu terapkan langsung pada studi kasus Simple POS di jobsheet praktikum
- Memetakan relasi `Category - Product` dan `Transaction - TransactionDetail - Product`, menghindari N+1 pada listing 2.500 transaksi, dan membuktikan paginasi pada 300 produk

Kode lengkap: github.com/se-polinema/simple-pos-ch05

Rangkuman

- Relasi (`hasOne` , `hasMany` , `belongsTo` , `belongsToMany`) dideklarasikan sekali di model, lalu dipakai lewat pemanggilan metode, akses properti, atau string di dalam `with()`
- Lazy loading memicu satu query tambahan per baris per relasi (masalah N+1); eager loading, termasuk notasi titik berjenjang, memuat relasi dalam jumlah query tetap, tidak bergantung jumlah baris
- ORM seperti Eloquent (memakai pola Active Record) memetakan class ke tabel dan objek ke baris, menggantikan query SQL mentah dengan method dan properti object
- `paginate()` membagi listing besar jadi halaman-halaman kecil; halaman berbeda bisa dibuktikan benar-benar berbeda dengan membandingkan ID hasil antar halaman

Referensi & Diskusi

Dokumentasi resmi Laravel (Eloquent, Relationships, Pagination)

Kode lengkap: `github.com/se-polinema/simple-pos`

Pertemuan berikutnya: Validasi & Keamanan Input